

VestiFlow — Guida operatore, proprietario e sviluppatore

Versione documento: 2.1 — Giugno 2026

Destinatari: operatori piattaforma VestiFlow (`isPlatformAdmin`), proprietario del prodotto, sviluppatori che mantengono il gestionale.

Visibilità: accessibile solo come **Guida tecnica** (`/app/admin/guide`) con account il cui email è in `PLATFORM_ADMIN_EMAILS` . I clienti negozio usano la voce **Guida** (`/app/guide`) con il manuale operativo del negozio.

Indice operatore

1. [Ruolo operatore piattaforma](#)
 2. [Architettura e stack](#)
 3. [Modello multi-tenant](#)
 4. [Struttura repository](#)
 5. [Ambiente di sviluppo locale](#)
 6. [Variabili d'ambiente](#)
 7. [Onboarding nuovo cliente \(tenant\)](#)
 8. [Autenticazione e ruoli](#)
 9. [Integrazione Shopify \(tecnica\)](#)
 10. [Supabase: database, Auth, Storage, RLS](#)
 11. [Dominio dati principale](#)
 12. [API e permessi tenant](#)
 13. [Import ed export CSV](#)
 14. [Scanner barcode](#)
 15. [Generazione guide \(HTML/PDF\)](#)
 16. [Build, test e CI](#)
 17. [Deploy produzione](#)
 18. [Sicurezza e checklist pre-release](#)
 19. [Troubleshooting tecnico](#)
 20. [Limitazioni note e roadmap](#)
-

1. Ruolo operatore piattaforma

L'**operatore piattaforma** (tu, come proprietario/sviluppatore) non coincide con il **Titolare** di un tenant negozio.

Concetto	Dove vive	Come si ottiene
Platform admin	Flag <code>isPlatformAdmin</code> nel profilo utente API	Email in <code>PLATFORM_ADMIN_EMAILS</code> (backend)
Titolare tenant	Ruolo <code>owner</code> nel DB tenant	Scelto al provisioning in Nuovo cliente
Ruoli negozio	<code>owner</code> , <code>admin</code> , <code>manager</code> , <code>clerk</code>	Provisioning in Nuovo cliente ; ulteriori utenti e permessi in Modifica cliente → Utenti del cliente

Cosa vede un platform admin in UI

Shell **dedicata** (non le schermate operative del negozio):

Voce menu	Route	Contenuto
Clienti	<code>/app/admin/clients</code>	Elenco tenant + form Nuovo cliente
Impostazioni	<code>/app/admin/account</code>	Profilo operatore, foto, MFA, tema
Guida tecnica	<code>/app/admin/guide</code>	Questo manuale

In topbar: **tema**, **avatar** (clic → Impostazioni), **Esci**. **Nessun** selettore sede né indicatore sync Shopify finché non entri in una **sessione assistenza** (vedi sotto).

Sessione assistenza al gestionale cliente

Flusso **preferito** per supporto tecnico: aprire il gestionale del cliente **senza password condivise**, con sessione tracciata e durata massima **2 ore**.

Dove in UI	Azione
Tabella Clienti registrati (colonna Assistenza)	Apri gestionale (assistenza)
Modifica cliente (<code>/app/admin/clients/:id</code>)	Pulsante Apri gestionale (assistenza) in header

Cosa succede:

1. L'API crea un record `SupportSession` (operatore → tenant target, scadenza `expiresAt`).
2. Il frontend salva l'ID sessione in `sessionStorage` e reindirizza a `/app/dashboard` del tenant.

3. Ogni richiesta API include l'header `X-Vestiflow-Support-Session` (interceptor `supportSessionInterceptor`).
4. `JwtAuthGuard` valida la sessione, **sovrascrive** `tenantId` e arricchisce il profilo con `supportSession`.
5. Durante la sessione l'operatore ha **permessi equivalenti ad admin tenant** (lettura + scrittura); `RolesGuard` bypassa i check ruolo se `request.supportSession` è attivo.
6. In shell compare un **banner fisso in basso**: «Assistenza — {nome cliente}» + **Esci dall'assistenza** (chiude la sessione e torna a `/app/admin/clients`).

Regole di sicurezza:

- Solo email in `PLATFORM_ADMIN_EMAILS` può avviare una sessione (`PlatformAdminGuard` + check in `SupportSessionService`).
- Non è possibile aprire assistenza su un tenant che contiene utenti platform admin.
- Una nuova sessione **chiude** eventuali sessioni attive dello stesso operatore.
- Alla scadenza (2 h) o su chiusura manuale, `endedAt` viene impostato; richieste successive con quell'ID → **401**.
- **Audit**: tabella `support_sessions` con operatore, tenant, `createdAt`, `expiresAt`, `endedAt`.

Alternativa (solo emergenza): login con credenziali del titolare del tenant. Evitare condivisione password: preferire sempre la sessione assistenza.

Backend

Endpoint admin protetti da `JwtAuthGuard` + `PlatformAdminGuard` (verifica email contro env):

Metodo	Path	Azione
POST	<code>/admin/tenants/:id/support-session</code>	Avvia sessione assistenza verso il tenant
DELETE	<code>/admin/support-sessions/current</code>	Termina sessione attiva dell'operatore corrente

Provisioning tenant: endpoint sotto `/api/v1/admin/tenants` (CRUD tenant).

2. Architettura e stack

Componente	Tecnologia	Ruolo
Frontend	Angular 21+ (standalone, signals, OnPush)	SPA/PWA gestionale
Hosting frontend	Firebase App Hosting (o equivalente static)	CDN + HTTPS
Backend API	NestJS	Business logic, Shopify OAuth, webhook
Hosting API	Railway (tipico)	Node 22, env secrets
Database	PostgreSQL via Supabase	Dati multi-tenant
Auth	Supabase Auth (JWT HS256)	Login email/password, MFA opzionale
Storage immagini	Supabase Storage bucket <code>product-media</code>	Upload media prodotti
Storage avatar	Supabase Storage bucket <code>user-avatars</code>	Foto profilo utente (public)
E-commerce	Shopify Admin API + webhook	Catalogo, stock, ordini, clienti
E-commerce alt.	TikTok Shop Open API + OAuth (parziale)	Catalogo push + giacenze — early stage

Regola: il frontend **non** contiene secret Shopify né service role Supabase. Tutto passa dall'API.

3. Modello multi-tenant

- Ogni **tenant** = un'azienda cliente = **un canale e-commerce** collegato (Shopify, TikTok Shop) oppure **solo gestionale**.
- Campo `channelProfile` sul tenant: `gestionale` | `shopify` | `tiktok_shop` — determina quali pannelli Integrazione compaiono in Impostazioni lato cliente.
- Isolamento dati: colonna `tenantId` su entità business; filtro obbligatorio lato API.
- **Location** ≠ **Store**: lo stock è per location (semantica Shopify); lo store è entità commerciale.
- **Variante** = unità minima inventario (SKU univoco interno).
- **Due shop Shopify distinti** → **due tenant** VestiFlow separati.
- **Più location nello stesso shop** → un tenant, N location sincronizzate.

4. Struttura repository

```
vestiflow/
├─ src/app/          # Frontend Angular
│  └─ core/          # Auth, guards, permissions, HTTP, support (sessione assistenza)
│  └─ shared/         # Componenti UI riutilizzabili
│  └─ features/       # Feature lazy-loaded (products, inventory, sales, admin, guide...)
│     └─ layout/      # Shell sidebar + topbar
├─ api/              # Backend NestJS
│  └─ src/admin/      # Provisioning tenant + support-session (platform admin)
│  └─ src/support/    # SupportSessionService, costanti TTL/header
│  └─ src/products/   # Catalogo, CSV import/export
│  └─ src/inventory/  # Giacenze, movimenti, CSV
│  └─ src/shopify/    # OAuth, sync, webhook, rate limit
│     └─ prisma/      # Schema e migration PostgreSQL
├─ docs/             # Guide Markdown + HTML/PDF
├─ public/guide/     # Guida utente in-app (pubblica)
├─ src/assets/guide-admin/ # Guida tecnica (solo route admin)
└─ scripts/          # generate-guide-*, check-rls, environment prod
```

Alias TypeScript: `@core/*`, `@shared/*`, `@features/*`, `@env/*`.

5. Ambiente di sviluppo locale

Requisiti

- Node.js **22** LTS (`.nvmrc`)
- npm (lockfile committato)
- Progetto Supabase (URL + anon key + service role per API)
- Account Shopify Partners (app custom) per test OAuth

Avvio

```
# Frontend (porta 4200)
npm install
npm start

# Backend (porta 3000)
cd api
cp .env.example .env # compilare variabili
npm install
npx prisma migrate dev
npm run start:dev
```

Shopify in locale

Shopify richiede URL **pubblici** per OAuth callback e webhook. Usa **ngrok** o **cloudflared**:

1. Tunnel verso `localhost:3000`
2. Imposta `SHOPIFY_APP_URL` e `FRONTEND_URL` nel `api/.env`
3. Aggiorna redirect URL nell'app Partners

PWA locale

```
npm run build:pwa:local
npm run serve:pwa
```

6. Variabili d'ambiente

Frontend (`.env` → **build**, valori **pubblici**)

Vedi `.env.example` in root:

Variabile	Significato
<code>VESTIFLOW_API_BASE_URL</code>	URL API produzione
<code>VESTIFLOW_SUPABASE_URL</code>	URL Supabase
<code>VESTIFLOW_SUPABASE_ANON_KEY</code>	Anon/publishable key
<code>VESTIFLOW_ENABLE_BARCODE_SCANNER</code>	Feature flag scanner
<code>VESTIFLOW_ENABLE_SHOPIFY</code>	Feature flag integrazione Shopify UI

Mai service role o secret Shopify nel frontend.

Backend (`api/.env`)

Vedi `api/.env.example` :

Variabile	Significato
DATABASE_URL / DIRECT_URL	PostgreSQL Supabase
SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY , SUPABASE_JWT_SECRET	Auth e admin DB
CORS_ORIGINS	Origini frontend consentite
(header CORS)	X-Vestiflow-Support-Session consentito per sessione assistenza
SHOPIFY_*	OAuth, scope, cifratura token, rate limit
PLATFORM_ADMIN_EMAILS	Email operatori (virgola)
SUPABASE_PRODUCT_MEDIA_BUCKET	Bucket immagini prodotto (product-media)
SUPABASE_USER_AVATARS_BUCKET	Bucket foto profilo (user-avatars)
TIKTOK_*	OAuth TikTok Shop, cifratura token, API base
FRONTEND_URL	Redirect post-OAuth

7. Onboarding nuovo cliente (tenant)

UI: /app/admin/clients (solo platform admin) — tabella **Clienti registrati** e pulsante **Nuovo cliente**.

Procedura creazione

1. **Identificazione** — nome commerciale, ragione sociale opzionale
2. **Anagrafica** — P.IVA, CF, sede, contatti (opzionali)
3. **Profilo canale** — **Solo gestionale**, **Shopify** o **TikTok Shop** (determina integrazioni visibili al cliente)
4. **Primo accesso** — ruolo VestiFlow (owner default), nome, email, **password iniziale** (scelta dall'admin)
5. **Piano sedi** — **Sedi incluse nel piano** (1–10): numero massimo di location operative in VestiFlow per questo contratto
6. **Setup** — nome negozio e location iniziale (opzionali; default «Negozio principale»)

Il backend crea: tenant, utente Supabase Auth con password, store, location (con `licensedInVf: true` solo se profilo **Solo gestionale**; con Shopify la licenza si assegna dopo sync + selezione cliente), profilo utente collegato.

Dopo il provisioning

Consegna credenziali al titolare **in modo sicuro** (email + password). Il titolare accede da `/login` e può cambiare password da Impostazioni o tramite «Password dimenticata».

Invito email (standby): il flusso con invito Supabase al titolare esiste nel codice ma è **disabilitato** finché non è attivo Supabase a pagamento con SMTP. Riattivazione: variabile Railway `SUPABASE_OWNER_EMAIL_INVITE=true` + configurazione redirect/template Supabase (vedi `commit/feature invito`).

Il titolare completa in base al profilo canale:

Profilo canale	Passi titolare
Shopify	MFA → OAuth Shopify → sync location → Sedi attive in VestiFlow → webhook → import
TikTok Shop	MFA → OAuth TikTok Shop → verifica location
Solo gestionale	MFA → catalogo e magazzino solo in VestiFlow

Modifica tenant esistente

Tabella **Clienti registrati** → click riga → `/app/admin/clients/:id`.

Modificabile: anagrafica, **profilo canale** (se nessuna integrazione attiva), **Sedi incluse nel piano** (1–10), nome titolare, negozio. **Email** e **ruolo** del primo utente sono **sola lettura** in UI (campi disabilitati).

Utenti del cliente: pannello in fondo a **Modifica cliente** (`admin-tenant-users-panel`). Permette di elencare, creare, aggiornare (ruolo, sede assegnata, permessi granulari) ed **eliminare** utenti tenant (icona cestino; il **titolare** non è eliminabile). I permessi partono dal preset del ruolo (`ROLE_DEFAULT_PERMISSIONS`) e possono essere personalizzati con checkbox; chiavi obsolete (es. `settings.integrations`) vengono filtrate al save (FE + BE). Dopo modifica permessi il cliente deve **re-login** o hard refresh per allineare menu e CTA.

Sedi attive (read-only): sotto il selettore piano compare l'elenco delle location attualmente **attive in VestiFlow** (`licensedInvf: true`), con nome in campi disabilitati (non è più editabile il «nome location» di provisioning). Se il cliente non ha ancora salvato la selezione, l'elenco può essere vuoto.

Concedi cambio sede: pulsante visibile quando la selezione è **bloccata** e non c'è già una concessione one-shot in corso. Imposta `locationSelectionChangeGranted=true` sul tenant: il cliente può modificare **una volta** le sedi attive in Impostazioni; al salvataggio la selezione si **riblocca** e la concessione viene azzerata.

Riduzione piano sedi: se abbassi **Sedi incluse nel piano** sotto il numero di sedi già attive, l'API **disattiva** le eccedenze (mantiene le più vecchie per `createdAt`, le altre passano a `licensedInvf: false`). Non sblocca la selezione: il cliente non può riattivare sedi diverse finché non gli concedi il cambio sede.

Da questa pagina (e dalla tabella clienti) puoi avviare **Apri gestionale (assistenza)** — vedi [§1 Sessione assistenza](#).

Per cambiare profilo canale con integrazione già connessa: il cliente deve **disconnettere** Shopify o TikTok da Impostazioni prima.

Eliminazione tenant (zona pericolosa)

In **Modifica cliente**, pannello **Zona pericolosa** → **Elimina cliente**: rimuove tenant, dati negozio, utenti e integrazioni. Operazione **irreversibile** con dialog di conferma.

API

Metodo	Path	Azione
GET	<code>/admin/tenants</code>	Lista tenant
POST	<code>/admin/tenants</code>	Crea tenant + primo utente
GET	<code>/admin/tenants/:id</code>	Dettaglio
PATCH	<code>/admin/tenants/:id</code>	Aggiorna anagrafica/setup
DELETE	<code>/admin/tenants/:id</code>	Elimina tenant e dati
POST	<code>/admin/tenants/:id/grant-location-selection-change</code>	Concede al cliente un round di modifica sedi attive
GET	<code>/admin/tenants/:id/users</code>	Lista utenti tenant
POST	<code>/admin/tenants/:id/users</code>	Crea utente tenant
PATCH	<code>/admin/tenants/:id/users/:userId</code>	Aggiorna utente (ruolo, permessi, sede)
DELETE	<code>/admin/tenants/:id/users/:userId</code>	Elimina utente (non owner)
POST	<code>/admin/tenants/:id/support-session</code>	Avvia sessione assistenza (2 h)
DELETE	<code>/admin/support-sessions/current</code>	Termina sessione assistenza attiva

Body create include `role` (`owner` | `admin` | `manager` | `clerk`), `channelProfile` (`gestionale` | `shopify` | `tiktok_shop`) e `licensedLocationCount` (default `1`, max `10`).

Migration DB: `0018_support_sessions` — tabella `support_sessions`; `0021_tenant_location_licensing` — `licensed_location_count`, `licensed_in_vf`; `0022_location_selection_lock` — blocco selezione + concessione admin. In produzione: `npm run prisma:deploy` (o equivalente Railway) prima di usare licensing sedi e assistenza.

8. Autenticazione e ruoli

Flusso auth

1. Login Supabase Auth (frontend)
2. JWT inviato all'API (`Authorization: Bearer`)
3. `JwtAuthGuard` valida firma con `SUPABASE_JWT_SECRET`
4. Profilo utente + `tenantId` + ruolo + `isPlatformAdmin` caricati

Sessione assistenza (auth)

Se la richiesta include l'header `X-Vestiflow-Support-Session` con un ID sessione valido e l'utente è platform admin:

1. `SupportSessionService.resolveActiveSession()` verifica ID, operatore, `endedAt` null, `expiresAt` futuro.
2. `JwtAuthGuard` imposta `request.tenantId` e `request.appUser.tenantId` al **tenant cliente** target.
3. Il profilo API espone `supportSession: { sessionId, targetTenantId, targetTenantName, expiresAt }`.
4. `RolesGuard` **non applica** vincoli ruolo tenant quando `request.supportSession` è presente.
5. Frontend: `tenant-permissions.util.ts` tratta l'operatore in assistenza come **admin tenant**; `isPlatformOperator()` è `false` durante la sessione (UI gestionale, non shell admin).

Persistenza client: `sessionStorage` (`SUPPORT_SESSION_STORAGE_KEY`). Restore al bootstrap in `app.config.ts`; interceptor HTTP allega l'header su ogni chiamata finché la sessione è attiva.

Permessi granulari tenant (UI + API)

Il **titolare** (`owner`) ha sempre accesso completo (`hasFullTenantAccess`); i permessi persistiti su `User.permissions` non lo limitano.

Per `admin`, `manager`, `clerk` valgono chiavi `TenantPermission` (FE: `tenant-permission.model.ts`, BE: `tenant-permission.constants.ts`). Preset per ruolo: `ROLE_DEFAULT_PERMISSIONS`. Normalizzazione e filtro chiavi legacy: `user-permissions.util.ts` (FE + BE).

Chiave	Gruppo	Uso principale
<code>inventory.view_all_locations</code>	inventory	Filtri giacenze/movimenti su tutte le sedi (azioni restano su sede operativa)
<code>inventory.manage</code>	inventory	Carichi, scarichi, trasferimenti, rettifiche, inventario fisico
<code>inventory.import_export</code>	inventory	CSV giacenze + sync giacenze Shopify
<code>catalog.manage</code>	catalog	CRUD prodotti
<code>catalog.import_export</code>	catalog	CSV catalogo + sync/import catalogo Shopify
<code>catalog.delete</code>	catalog	Delete prodotto
<code>supplier_orders.manage</code>	orders	CRUD ordini fornitore
<code>supplier_orders.receive</code>	orders	Ricezione merce
<code>retail.register</code>	orders	POST <code>/inventory/retail-scans</code> , pagina Registra vendita
<code>reports.view</code>	reports	Dashboard e Report
<code>reports.export</code>	reports	Export CSV + sync vendite/clienti Shopify
<code>settings.company</code>	settings	GET <code>/tenant/company</code> , pannello Sede fisica
<code>customers.view</code> / <code>customers.manage</code>	customers	Lista clienti / gestione (se prevista)

Rimosso: `settings.integrations` — filtrato al save admin e in normalizzazione profilo.

Solo titolare (`hasFullTenantAccess` , `@Roles(owner)` su controller OAuth): connessione/disconnessione Shopify e TikTok, sync location, salvataggio sedi attive, shop-change wizard.

Guard frontend: `tenantPermissionGuard` + `data.tenantPermissions` sulle route; sidebar filtrata da `tenant-permissions.util.ts` . **Guard backend:** `TenantPermissionsGuard` + decorator `@RequirePermissions(...)` ; `RolesGuard` per operazioni owner-only.

Scope sedi: `OperationalLocationsService` (FE) distingue `locations` (lettura/filtri), `writeLocations` (azioni topbar) e `transferTargetLocations` . BE: `user-location-scope.util.ts` → `assertUserCanAccessLocation` con scope `read` / `write` / `transferDestination` . Topbar: commesso/manager con sede assegnata → etichetta fissa; titolare/admin con più sedi → select.

Admin utenti: `AdminTenantUsersService` (`api/src/admin/admin-tenant-users.service.ts`); UI `admin-tenant-users-panel` in Modifica cliente.

Sync Shopify vs permessi:

Operazione	Permesso
Import/sync catalogo	<code>catalog.import_export</code>
Sync giacenze	<code>inventory.import_export</code>
Sync vendite/clienti	<code>reports.export</code>

Route Angular sensibili: `tenantPermissionGuard` (sostituisce il vecchio guard solo-ruolo dove applicabile).

Foto profilo utente

Metodo	Path	Azione
POST	<code>/auth/avatar</code>	Upload foto (multipart <code>file</code>)
DELETE	<code>/auth/avatar</code>	Rimuove foto e file su Storage

Validazione: JPEG/PNG/WebP, max 2 MB. Bucket `user-avatars` (public). Dopo upload/delete invalida cache profilo JWT (`AuthProfileCacheService.invalidate`).

UI: **Impostazioni** → **Profilo** (tenant) e **Impostazioni** operatore (`/app/admin/account`). Avatar in topbar cliccabile → Impostazioni.

9. Integrazione Shopify (tecnica)

Ownership sync (riepilogo)

Entità	Owner	Note
Catalogo prodotti VestiFlow (catalogOrigin=vestiflow)	VestiFlow	CRUD completo; push al save; delete write-through verso Shopify se shopifyProductId
Catalogo prodotti Shopify (catalogOrigin=shopify)	Shopify	Pull import/webhook; in VF solo PATCH stagione + purchasePriceMinor ; no delete/sync manuale/media
Clienti, ordini online	Shopify	Read-only in VF
Giacenze	Condiviso	VF: carichi/rettifiche; Shopify: vendite
Location	Shopify master	Import + mapping; cleanup sedi stale; operatività VF via licensedInVf + piano tenant
Ordini fornitori	Solo VestiFlow	—
Anagrafica tenant	Solo VestiFlow (admin)	GET /tenant/company read-only in UI Sede fisica

Origine catalogo (catalogOrigin / shopifyCatalogLinkKind)

Campi su Product (migration 0016 + 0017):

Valore	Significato	Impostato quando
catalogOrigin=vestiflow	Il gestionale possiede i campi catalogo condivisi	Create in VF, import CSV, push verso Shopify (linkKind=pushed)
catalogOrigin=shopify	Shopify Admin possiede il catalogo	Import pull (linkKind=imported) o backfill legacy
shopifyCatalogLinkKind=imported	Collegato da import Shopify → VF	ShopifyProductPullService
shopifyCatalogLinkKind=pushed	Creato in VF e inviato al canale	Create/push ShopifyProductPushService , import CSV

Logica centralizzata in `api/src/products/catalog-origin.util.ts`:

- `isVestiflowCatalogOwner()` — esclude import Shopify e legacy con link alla creazione; include push e prodotti con media locale Supabase
- `shouldSkipShopifyCatalogImport()` — pull/webhook **non sovrascrivono** catalogo VF-owned
- `assertShopifyCatalogUpdateAllowed()` — su `shopify`: blocca mutazioni catalogo; consente `season` + `purchasePriceMinor`
- `assertShopifyCatalogDeleteAllowed()` / `assertShopifyCatalogManualSyncAllowed()` / `assertShopifyCatalogMediaMutationAllowed()`

Backfill tenant esistenti (dopo deploy migration):

```
cd api && npm run backfill:catalog-origin          # dry-run
cd api && npm run backfill:catalog-origin:apply    # scrivo su DB
```

UI tenant: colonna **Fonte** in lista prodotti; badge `Fonte: VestiFlow` / `Fonte: Shopify` in dettaglio; form in modalità **Modifica dati operativi** se `catalogOrigin=shopify` (`catalog-origin.util.ts` FE + `product-form` / `product-detail`).

Taxonomy Shopify e metafield di categoria

Picker e attributi categoria nel form prodotto (step **Dati generali**).

Layer	Percorso / componente
API	<code>GET /shopify/taxonomy/categories</code> , <code>GET /shopify/taxonomy/category-attributes?categoryId=</code>
BE	<code>ShopifyTaxonomyService</code> , <code>ShopifyCategoryMetafieldsService</code> , util <code>shopify-category-metafields.util.ts</code>
FE picker categoria	<code>shopify-taxonomy-picker</code>
FE attributi	<code>shopify-category-attributes</code> (in <code>product-general-step</code>)
Persistenza	<code>Product.shopifyCategoryMetafields</code> (JSON), <code>shopifyTaxonomyCategoryId</code> / <code>FullName</code>

Modello `shopifyCategoryMetafields` : array di oggetti con campi `attributeId` , `attributeName` , `namespace` , `key` , `metafieldType` e array `values` . Ogni attributo può avere **più valori** in `values` (allineato a Shopify).

UI multi-valore: se `metafieldType` inizia con `list`. → `SelectMenuComponent` in modalità `multiple` (`isShopifyCategoryMetafieldMultiValue` in `shopify-category-metafield.util.ts`). Altrimenti select singola.

Push: `ShopifyProductPushService` → `pushCategoryMetafields` serializza i GID taxonomy (`serializeTaxonomyValueListGids`) o crea metaobject per tipo `list.metaobject_reference`. Import/pull popola lo stesso JSON da metafield prodotto Shopify.

Cambio negozio e purge dati Shopify

Modulo `ShopifyShopChangeService` + wizard FE `shopify-shop-change-wizard`.

Endpoint	Metodo	Ruolo	Scopo
<code>/shopify/shop-change/preview</code>	GET	owner	Conteggi dati collegati a Shopify + blockers (es. ordini fornitori aperti su location Shopify)
<code>/shopify/shop-change/purge</code>	POST	owner	Rimuove dati importati/syncati da Shopify (prodotti, varianti, clienti, ordini vendita, location collegate, giacenze/movimenti associati)
<code>/shopify/connection</code>	DELETE	owner	Disconnessione OAuth (token revocato); non purge catalogo

Flussi UI:

- **Cambia negozio** — wizard mode `shop-change`: preview → opzione purge → conferma dominio → disconnect → redirect OAuth nuovo shop
- **Disconnetti e rimuovi dati** — wizard mode `disconnect`: preview → purge opzionale → disconnect
- **Disconnetti Shopify** — solo DELETE connection, dati locali restano

OAuth guard: tentativo di collegare un dominio diverso da quello attivo senza purge precedente → errore esplicito (evita fork silenzioso tra shop).

Cosa NON cancella il purge: ordini fornitori (salvo blocker se legati a location Shopify), anagrafica tenant (**Sede fisica**), utenti tenant, movimenti non legati a entità Shopify rimosse.

Eliminazione prodotto

`ProductsService.delete()`:

1. `assertShopifyCatalogDeleteAllowed()` — se `catalogOrigin=shopify` → **409** con messaggio utente (elimina da Shopify Admin)
2. Verifica assenza movimenti stock sul prodotto
3. Se `shopifyProductId` presente e `catalogOrigin=vestiflow` → `ShopifyProductPushService.deleteProduct()` **prima** del delete DB
4. Errori mappati: `not_connected`, `missing_write_products_scope`, `shopify_error` → **422** con messaggio utente; DB intatto

Scope richiesto per delete write-through: `write_products` (incluso in `SHOPIFY_SCOPES` default).

FE: pulsante **Elimina** e **Sincronizza con Shopify** nascosti se `catalogOrigin=shopify`.

Sync location — comportamento post-fix

`ShopifyLocationSyncService`:

- Importa/aggiorna location da Shopify; **non** collega più automaticamente la sede onboarding `LOC-01` al primo match Shopify
- `buildLinkedLocationData` aggiorna il **nome** da Shopify a ogni re-sync
- `cleanupStaleShopifyLocations` rimuove location non più presenti su Shopify API
- `removeEmptyOnboardingLocation` elimina sede temporanea onboarding vuota e non collegata dopo sync
- Dopo sync/disconnect/purge: `ShopifyConnectionRefreshService.notifyInvalidated()` → shell ricarica location e topbar

FE: `filterLocationsForTopbar` e `OperationalLocationsService` espongono **solo** location con `licensedInVf: true` e `isActive: true` (nasconde anche sede onboarding locale quando Shopify è connesso).

Licensing sedi e blocco selezione

Modulo `LocationLicensingService` (`api/src/inventory/location-licensing.service.ts`).

Concetto	Campo / comportamento
Piano contrattuale	<code>Tenant.licensedLocationCount</code> (1–10), impostato in create/edit admin
Sede operativa VF	<code>Location.licensedInVf</code> — solo queste compaiono in topbar, magazzino, movimenti, export CSV giacenze
Blocco selezione	<code>Tenant.locationSelectionLocked</code> — dopo primo salvataggio cliente (<code>PUT /inventory/locations/licensed</code>)
Concessione admin	<code>Tenant.locationSelectionChangeGranted</code> — one-shot; azzerata al salvataggio cliente
API summary	<code>canChangeLicensedLocations = !locked granted</code> in <code>GET /tenant/company</code> e <code>GET /admin/tenants/:id</code>

Endpoint	Metodo	Ruolo	Scopo
<code>/inventory/locations/licensed</code>	PUT	owner	Cliente salva elenco <code>locationIds</code> attive ($\leq$ piano); default <code>lockAfterSave: true</code>
<code>/admin/tenants/:id/grant-location-selection-change</code>	POST	platform admin	Sblocca un round di modifica per il tenant

Regole business:

- **403** se il cliente tenta di salvare con selezione bloccata e senza concessione.
- **Auto-licenza singola sede Shopify:** se piano = 1 e Shopify espone 1 sola location attiva, `tryAutoLicenseSingleShopifyLocation()` pre-seleziona senza bloccare (`lockAfterSave: false` , `bypassSelectionLock: true`); il blocco scatta al primo salvataggio esplicito del cliente quando applicabile.
- **Trim admin:** abbassare `licensedLocationCount` disattiva le sedi in eccesso (FIFO per `createdAt`), **senza** sbloccare la selezione.
- **Scope inventario:** `licensed-location-scope.util.ts` filtra query movimenti, giacenze, inventario fisico, retail-scans su sedi licenziate.

FE tenant: `location-licensing-panel` in Impostazioni (Shopify); messaggi blocco/concessione in fondo al pannello.

FE admin: `edit-client` — elenco `activeLocations` read-only + **Concedi cambio sede**.

Migration: `0021_tenant_location_licensing` , `0022_location_selection_lock` (backfill: tenant con sedi già `licensedInVf` → `locationSelectionLocked=true`).

Anagrafica tenant (Sede fisica)

Endpoint	Metodo	Scopo
<code>/tenant/company</code>	GET	Dati commerciali tenant (name, storeName, PIVA, indirizzo, contatti) — read-only lato cliente

Popolati al provisioning admin (`create-client`). UI: `tenant-client-card` in Impostazioni.

OAuth

- Scope default in `SHOPIFY_SCOPES` (`.env.example`)
- Token cifrati at rest (`SHOPIFY_TOKEN_ENCRYPTION_KEY`)
- Diagnostica scope in Impostazioni: richiesti vs concessi

Webhook

Registrati con **Attiva aggiornamenti automatici**. Idempotenza lato backend per eventi duplicati/out-of-order.

Limiti Shopify Admin API — concetto

Shopify **non addebita le chiamate API** a consumo. Il piano del negozio definisce **quanto velocemente** puoi chiamare le Admin API prima di essere **rallentato** (HTTP **429 Too Many Requests** / throttling). Non esiste un contatore “a pagamento per chiamata”.

VestiFlow usa oggi soprattutto la **REST Admin API** (prodotti, location, inventario, immagini).

Documentazione ufficiale Shopify:

- [Limiti API \(panoramica\)](https://shopify.dev/docs/api/usage/limits) (https://shopify.dev/docs/api/usage/limits)
- [Rate limit REST Admin API](https://shopify.dev/docs/api/admin-rest/usage/rate-limits) (https://shopify.dev/docs/api/admin-rest/usage/rate-limits)

Fasce tipiche REST (piano negozio → velocità massima):

Piano Shopify (indicativo)	REST Admin API
Basic / Grow (Standard)	~ 2 richieste/sec , bucket 40
Advanced	~ 4 richieste/sec , bucket 80
Plus	limiti più alti (fascia Enterprise)

Il header `X-Shopify-Shop-API-Call-Limit` (es. `32/40`) indica quanto del bucket è consumato.

Cosa fa VestiFlow quando si avvicina o supera il limite

Implementazione in `api/src/shopify/`:

Componente	Ruolo
<code>ShopifyRateLimiterService</code>	Throttle per shop prima di ogni richiesta outbound
<code>ShopifyAdminClient.request()</code>	Applica throttle, legge header bucket, retry su 429
<code>ShopifyProductPullService</code>	Mutex import catalogo (un import per tenant alla volta, process-local)

Comportamento:

1. **Intervallo minimo** tra richieste (default 550 ms $\approx$ 1,8 req/s, sotto il limite Basic 2/s).
2. Se `X-Shopify-Shop-API-Call-Limit` $\geq$ soglia (**85%** del bucket), pausa **1 s** prima delle richieste successive.

3. Su **HTTP 429**: legge `Retry-After`, backoff esponenziale, fino a **5** retry; poi errore 429 con messaggio utente: «*Shopify ha limitato temporaneamente le richieste API...*».
4. **Import catalogo**: enrichment leggero (`skipRemoteMetadata: true`, niente costi varianti extra) per ridurre chiamate; **non avviare due import** sullo stesso tenant in parallelo.
5. **Webhook** prodotti/ordini/giacenze: **0 chiamate outbound** — Shopify invia i dati a VestiFlow.

Variabili env (REST outbound):

Variabile	Default	Effetto
<code>SHOPIFY_API_MIN_INTERVAL_MS</code>	550	Pausa minima tra richieste
<code>SHOPIFY_API_MAX_RETRIES</code>	5	Retry su HTTP 429
<code>SHOPIFY_API_BUCKET_HIGH_WATERMARK</code>	0.85	Pausa se secchio pieno
<code>SHOPIFY_API_BUCKET_PAUSE_MS</code>	1000	Durata pausa secchio

Nota deploy: il rate limiter Shopify è **process-local**. Con più repliche Railway ogni istanza ha il proprio contatore (comportamento conservativo, non centralizzato).

Chiamate indicative per operazione:

Operazione	Chiamate Shopify (ordine di grandezza)	Note
Import catalogo (Shopify → VF)	~4 per 1000 prodotti (pagine da 250)	Lento con cataloghi grandi: normale ; imposta <code>catalogOrigin=shopify</code>
Webhook prodotti/giacenze/ordini	0 outbound	Event-driven; skip catalogo se <code>vestiflow-owned</code>
Push prodotto al save (VF → Shopify)	1–N per prodotto	Solo <code>catalogOrigin=vestiflow</code> ; imposta <code>linkKind=pushed</code>
Delete prodotto (VF → Shopify)	1	Solo <code>catalogOrigin=vestiflow</code> + <code>shopifyProductId</code>
Purge dati Shopify	0 outbound (delete DB locale)	Dopo purge, riconnessione OAuth
Push giacenza dopo movimento	~1 per movimento	Write-through inventario
Sync location	1	Trascurabile

Cosa fare se un cliente vede errori 429:

1. Non ripremere import/sync in loop — attendere 1–2 minuti e **un solo** retry.
2. Evitare import catalogo + sync massivo contemporanei sullo stesso negozio.
3. In Railway, verificare env rate limit (non abbassare `SHOPIFY_API_MIN_INTERVAL_MS` sotto 500 ms su piani Basic).
4. Cataloghi molto grandi: l'import può richiedere diversi minuti; il backend riprova da solo fino al limite retry.

Non ancora implementato (roadmap): coda lavori persistente in background per bulk multi-tenant; oggi le operazioni massicce sono serializzate per shop/tenant ma restano sincrone nella request HTTP.

Rate limiting (REST Admin API) — riferimento rapido env

Vedi tabella variabili sopra. Valori in `api/.env.example`.

Import catalogo: enrichment ridotto (`skipRemoteMetadata`) + mutex un import per shop.

Troubleshooting `read_products`

1. App Partners: versione attiva include `read_products` , `read_inventory`
2. `SHOPIFY_API_KEY` Railway = Client ID app
3. Disinstalla app dal negozio Shopify
4. Disconnetti + riconnetti in VestiFlow
5. Verifica riga **Catalogo prodotti** → **Lettura** in Impostazioni

Protected customer data

Ordini/clienti webhook possono richiedere approvazione Partners. Catalogo/giacenze non dipendono da questo.

Integrazione TikTok Shop (tecnica)

Stato: early / parziale. OAuth + push catalogo (create/update) + push giacenze dopo movimenti VF. Non implementati: import da TikTok, webhook, vendite/clienti, parità funzionale con Shopify. Aggiornare questa sezione quando l'integrazione sarà completa.

Modulo `api/src/tiktok/` (OAuth, connessione, sync catalogo e inventory push).

Aspetto	Dettaglio
Profilo tenant	Solo tenant <code>channelProfile = tiktok_shop</code> vedono pannello Impostazioni
OAuth	Partner Center → env <code>TIKTOK_APP_KEY</code> , <code>TIKTOK_APP_SECRET</code> , <code>TIKTOK_SERVICE_ID</code>
Token	Cifrati at rest (<code>TIKTOK_TOKEN_ENCRYPTION_KEY</code>)
Sync scope	Push prodotti create/update; giacenze dopo carico/scarico VF
Non in scope	Vendite e clienti TikTok in UI (a differenza di Shopify)
Permessi UI	Collegamento OAuth: solo <code>owner</code> (<code>canManageTikTokConnection</code> / <code>hasFullTenantAccess</code>)

Callback OAuth: query `?tiktok=connected|error|disconnected` su ritorno frontend Impostazioni.

Variabili aggiuntive in `api/.env.example`: `TIKTOK_APP_URL`, `TIKTOK_OAUTH_CALLBACK_URL`, URL API/auth opzionali.

10. Supabase: database, Auth, Storage, RLS

Prisma

- Schema: `api/prisma/schema.prisma`
- Migration: `npx prisma migrate dev` (locale), deploy in CI/CD API

RLS obbligatoria

Ogni tabella business deve avere RLS attiva. Ci esegue `scripts/check-rls.mjs` (workflow `.github/workflows/security.yml`) con anon key: **zero righe** leggibili.

Storage

- Bucket `product-media` public per immagini prodotto
- Bucket `user-avatars` public per foto profilo utente
- Upload via API (service role); avatar: `UserAvatarService`, prodotti: pipeline esistente

Auth

- Utenti creati al provisioning (`admin-tenants.service`)
- MFA: Supabase TOTP, UI in Impostazioni

11. Dominio dati principale

Entità	Note
Tenant	Azienda cliente; <code>licensedLocationCount</code> , <code>locationSelectionLocked</code> , <code>locationSelectionChangeGranted</code>
User	Profilo app, <code>tenantId</code> , ruolo, <code>permissions[]</code> (permessi granulari; ignorati per owner)
Store / Location	Store commerciale; location per stock; <code>licensedInvf</code> = sede operativa nel piano VF
Product / ProductVariant	Opzioni generiche; SKU univoco; <code>catalogOrigin</code> , <code>shopifyCatalogLinkKind</code> , <code>shopifyCategoryMetafields</code> , taxonomy categoria
InventoryLevel	<code>variantId</code> × <code>locationId</code> , stati quantità. Riga creata al primo movimento/sync/rettifica/import; senza riga il prodotto non compare nel browse Giacenze. Con <code>GET /inventory/levels?search=...</code> l'API include anche varianti match senza riga (quantità 0, id sintetico <code>virtual:{variantId}:{locationId}</code>).
StockMovement	Audit trail obbligatorio; origine <code>vestiflow_pos</code> per vendite/storni al banco (tutti i profili canale)
SupplierOrder	Solo VF
SalesOrder / Customer	Import Shopify, read-only UI; assenti in UI profilo Solo gestionale
ShopifyConnection	Token, scope, stato sync per tenant
SupportSession	Audit sessioni assistenza: <code>operatorUserId</code> , <code>targetTenantId</code> , <code>expiresAt</code> , <code>endedAt</code>

Denaro: **interi minor units** (`Money.amountMinor`), mai float.

12. API e permessi tenant

Base URL: `/api/v1`. Header `Authorization` obbligatorio (salvo health).

Enforcement permessi (NestJS)

- `TenantPermissionsGuard` + `@RequirePermissions(...)` su endpoint sensibili (catalogo, magazzino, export, ordini fornitori).
- `RolesGuard` + `@Roles('owner')` su OAuth Shopify/TikTok, licensing sedi, shop-change.

- Normalizzazione permessi utente: `api/src/auth/user-permissions.util.ts` (`normalizeStoredPermissions`, rimozione chiavi obsolete).
- Scope location su mutazioni inventario: `api/src/inventory/user-location-scope.util.ts`.

Rate limiting API VestiFlow (NestJS)

Separato dai limiti Shopify: protezione anti brute-force / DoS sull'**API propria**.

Parametro	Valore	Dove
Limite globale	300 richieste / minuto / IP	<code>ThrottlerModule</code> in <code>api/src/app.module.ts</code>
Guard	<code>ThrottlerGuard</code> su tutte le route	Eccetto route marcate <code>@Public()</code>
Webhook Shopify inbound	Esclusi dal throttling globale	<code>shopify-webhooks.controller.ts</code> — non perdere eventi legittimi

Se un client supera 300 req/min riceve **429**; il frontend lo mappa in `AppError` kind `rate_limited` («Troppe richieste, riprova tra poco»).

Pattern service NestJS:

- Validazione DTO (`class-validator`)
- Filtro `tenantId` da JWT
- Errori → messaggi utente in italiano dove applicabile

Frontend: `HttpClient` + interceptor auth/error; mock disabilitati in prod.

13. Import ed export CSV

Catalogo prodotti

Export	<code>GET /products/export/csv</code> — formato Shopify CSV
Import	<code>POST /products/import/csv</code> — preview + commit; imposta <code>catalogOrigin=vestiflow</code> , <code>linkKind=pushed</code>
UI	Prodotti → Esporta / Importa CSV
Permessi	<code>catalog.import_export</code> (import/sync); <code>catalog.manage</code> (CRUD); <code>catalog.delete</code> (delete); export anche <code>reports.export</code>

Giacenze

Lista	GET /inventory/levels — paginata; query <code>locationId</code> , <code>search</code> , <code>lowStockOnly</code> , <code>page</code> , <code>pageSize</code> . Solo location licenziate e attive . Con <code>search</code> : espande varianti trovate (SKU/barcode/nome) anche senza riga in DB, una riga per sede con quantità 0. Senza <code>search</code> : solo righe esistenti in <code>inventory_levels</code> .
Sedi attive	PUT /inventory/locations/licensed — body { <code>locationIds</code> : <code>string[]</code> }; solo owner ; vedi §9 Licensing sedi
Export	GET /inventory/levels/export/csv
Import	POST /inventory/levels/import/csv — rettifiche
Colonne import	SKU, Location (nome esatto), Disponibile
UI	Magazzino → Giacenze
Permessi	lista: <code>autenticato</code> ; export/import/sync giacenze: <code>inventory.import_export</code> ; export CSV anche <code>reports.export</code> ; mutazioni: <code>inventory.manage</code>

Vendite e clienti

Export CSV dalle liste (filtri rispettati). Sync manuale Shopify vendite/clienti: `reports.export`.

Connessione OAuth Shopify/TikTok: solo `owner`.

Ordini fornitori

Metodo	Path	Azione	Permessi
GET	/supplier-orders	Lista paginata (ricerca, filtro stato)	autenticato
GET	/supplier-orders/:id	Dettaglio	autenticato
POST	/supplier-orders	Crea ordine (bozza o inviato)	supplier_orders.manage
PATCH	/supplier-orders/:id	Aggiorna bozza (righe sostituite integralmente)	supplier_orders.manage
POST	/supplier-orders/:id/send	Bozza → inviato	supplier_orders.manage
POST	/supplier-orders/:id/cancel	Annulla (solo bozza o inviato, non ancora ricevuto)	supplier_orders.manage
DELETE	/supplier-orders/:id	Elimina ordine annullato (righe in cascade)	supplier_orders.manage
POST	/supplier-orders/:id/receive	Ricezione merce + movimenti load + push inventario	supplier_orders.receive

Service: `SupplierOrdersService` (`api/src/supplier-orders/`). Ricezione in transazione atomica con `StockMovement` .

UI tenant: form ordine con `select-menu` searchable (fornitore, variante), `date-input` per data attesa, subtotale riga calcolato; dettaglio con **Elimina ordine** se `status=cancelled` . Lista prodotti: stampa etichette multi-select (`ProductLabelPrintService.triggerDirectPrintMany`).

Vendita al banco (tutti i profili canale)

Endpoint dedicato per **doppia scansione** al banco: decremento/incremento stock senza creare `SalesOrder` . Disponibile per `gestionale` , `shopify` e `tiktok_shop` .

Metodo	Path	Body / azione	Permessi
POST	/inventory/retail-scans	{ code, locationId, action: 'sale' 'return' } — qty fissa 1 per scansione	retail.register

Backend: `InventoryService.registerRetailScan()` (`api/src/inventory/inventory.service.ts`).

- `action: sale` → `StockMovement` tipo `sale` , origine `vestiflow_pos`
- `action: return` → `StockMovement` tipo `return` , origine `vestiflow_pos`
- Lookup variante: stessa semantica di `GET /products/variants/by-code/:code` (SKU o barcode)

- Vendita rifiutata se `available < 1` sulla location
- Post-scan: `ChannelSyncFacade.pushInventoryLevels()` (Shopify/TikTok se connessi)
- Migration enum: `0019_vestiflow_pos_movement_origin` → `MovementOrigin.vestiflow_pos`

I tipi `sale` e `return` **non** sono selezionabili nel form manuale **Registra movimento** (solo via retail-scans).

Frontend:

Rotta	Componente	Guard / note
<code>/app/sales/register</code>	<code>RetailSaleRegisterComponent</code>	<code>retailSalesRegisterGuard</code> — tutti i profili canale
<code>/app/sales</code>	lista ordini Shopify	<code>salesHistoryGuard</code> — redirect gestionale → register

Navigazione shell (`shell-layout.component.ts`):

- `showRetailSalesRegister(profile)` — Registra vendita per gestionale, Shopify, TikTok
- `showSalesOrderHistory(profile)` — **Vendite** solo Shopify
- Profilo Shopify: entrambe le voci in sidebar; `activeRouteExclude` evita doppia evidenziazione su `/app/sales/register`
- Service HTTP: `InventoryService.registerRetailScan()`
(`src/app/features/inventory/services/inventory.service.ts`)
- Label origine movimento: **Vendita negozio** (`inventory-labels.util.ts` → `MovementOrigin.VestiflowPos`)

Sidebar: evidenziazione sezione su sotto-route

Componente `shared/components/app-sidebar` + util `shared/utils/nav-link-active.util.ts`.

Ogni `NavItem` può definire `activeRoutePrefix` (es. `/app/inventory` per link a `/app/inventory/lookup`) così la voce resta evidenziata su tab e sotto-pagine (Giacenze, Movimenti, dettaglio prodotto, ecc.).
Opzionale `activeRouteExclude` per escludere route sorelle (Vendite vs Registra vendita).

14. Scanner barcode e componenti UI condivisi

Scanner barcode

- Componente: `shared/components/barcode-scanner` (BarcodeDetector API)

- Flag: `VESTIFLOW_ENABLE_BARCODE_SCANNER`
- Schermate: Cerca giacenza, Giacenze, Registra movimento, **Registra vendita** (vendita + storno), Inventario fisico, Prodotti
- Lookup API: `GET /products/variants/by-code/:code` (SKU o barcode esatto)
- **Pistola USB (keyboard wedge)**: nessuna integrazione dedicata — input HTML + Invio; usata su Registra vendita e altre schermate con campo codice
- Fallback iOS: input manuale

Date e select (form / filtri)

- `shared/components/date-input` — calendario custom al posto di `<input type="date">`; usato in **Report** (filtri periodo) e **Ordini fornitori** (data attesa)
- `shared/components/select-menu` — prop opzionale `searchable` con filtro su label e detail (`shared/utils/select-menu-filter.util`); usata su select **fornitore** e **variante** nel form ordine fornitore; opzioni variante a due righe (titolo + SKU via `variant-select-menu.util`)

15. Generazione guide (HTML/PDF)

```
npm run docs:guide:all
```

Genera:

Output	Contenuto	Audience
<code>public/guide/content.html</code>	Solo guida utente	Tutti (<code>/app/guide</code>)
<code>public/guide/vestiflow-guida.pdf</code>	PDF utente	Download pubblico in-app
<code>src/assets/guide-admin/content-tecnica.html</code>	Solo guida operatore/tecnica	Platform admin
<code>src/assets/guide-admin/vestiflow-guida-tecnica.pdf</code>	PDF tecnico	Download admin
<code>docs/GUIDA-*.html/pdf</code>	Sorgenti in repo	Documentazione offline

Markdown sorgenti:

- `docs/GUIDA-UTENTE-VESTIFLOW.md`
- `docs/GUIDA-OPERATORE-VESTIFLOW.md`

Dopo modifica guide: **rigenerare** e committare HTML/PDF prima del deploy.

16. Build, test e CI

Frontend

```
npm run lint
npm run build      # pre-push hook
ng test            # Vitest
```

Backend

```
cd api && npm run lint && npm run build
cd api && npm run test
```

Copertura automatica — Shopify shop change / location / delete

Area	File test
Purge / preview shop change	<code>api/src/shopify/shopify-shop-change.service.spec.ts</code>
Sync location + cleanup onboarding/stale	<code>api/src/shopify/shopify-location-sync.service.spec.ts</code>
Delete prodotto write-through Shopify	<code>api/src/products/products.service.spec.ts</code>
Guard <code>catalogOrigin</code> (update/delete/sync/media)	<code>api/src/products/catalog-origin.util.spec.ts</code>
Wizard UI (anteprima, conferma, disconnect)	<code>src/app/features/integrations/shopify/components/shopify-shop-change-wizard/*.spec.ts</code>
HTTP client shop change / sync location	<code>src/app/features/integrations/shopify/services/shopify-connection.service.spec.ts</code>
E2E wizard (anteprima, step conferma, annulla)	<code>e2e/shopify.spec.ts</code>
Retail scan API (sale/return, profili, push canale)	<code>api/src/inventory/inventory.service.spec.ts</code> , <code>inventory.controller.spec.ts</code>
Guard vendite / retail register	<code>src/app/features/sales-orders/guards/retail-sales.guard.spec.ts</code>
Pagina Registra vendita	<code>src/app/features/sales-orders/retail-sale-register.component.spec.ts</code>
HTTP client retail-scans	<code>src/app/features/inventory/services/inventory.service.spec.ts</code>
Profilo canale / label origine movimento	<code>tenant-channel-profile.model.spec.ts</code> , <code>inventory-labels.util.spec.ts</code>
Evidenza sidebar su sotto-route	<code>src/app/shared/utils/nav-link-active.util.spec.ts</code>
Licensing sedi + blocco selezione (BE)	<code>api/src/inventory/location-licensing.service.spec.ts</code>
Scope query su sedi licenziate	<code>api/src/inventory/licensed-location-scope.util.spec.ts</code>
Admin grant + trim piano / activeLocations	<code>api/src/admin/admin-tenants.service.spec.ts</code>

Area	File test
Pannello Sedi attive (FE)	<code>src/app/features/settings/components/location-licensing-panel/*.spec.ts</code>
Util lock/grant UI	<code>src/app/core/utills/location-selection-lock.util.spec.ts</code> , <code>admin-location-selection.util.spec.ts</code>
Summary licensing in tenant company	<code>src/app/features/settings/models/tenant-company.model.spec.ts</code> , <code>tenant-company.service.spec.ts</code>
Permessi tenant (FE util + guard)	<code>src/app/core/permissions/tenant-permissions.util.spec.ts</code> , <code>tenant-permission.guard.spec.ts</code>
Permessi utente / legacy keys	<code>src/app/core/permissions/user-permissions.util.spec.ts</code> , <code>api/src/auth/user-permissions.util.spec.ts</code>
Scope sedi utente (FE + BE)	<code>src/app/core/utills/user-location-scope.util.spec.ts</code> , <code>api/src/inventory/user-location-scope.util.spec.ts</code>
Topbar sede fissa vs select	<code>src/app/shared/components/app-topbar/app-topbar.component.spec.ts</code>
Admin save utenti / filtro permessi	<code>api/src/admin/admin-tenant-users.service.spec.ts</code>
E2E permessi commesso (base)	<code>e2e/permissions.spec.ts</code> — variabili <code>E2E_CLERK_*</code> in <code>.env</code>
E2E permessi owner/admin	<code>e2e/permissions-owner.spec.ts</code> — <code>E2E_USER_*</code> + sessione setup
E2E permessi granulari (catalog vs inventory import)	<code>e2e/permissions-granular.spec.ts</code> — <code>E2E_CLERK_CATALOG_IMPORT_*</code> , <code>E2E_CLERK_INVENTORY_IMPORT_*</code>
Provision utenti E2E granulari	<code>npm run provision:e2e-users</code> → <code>api/scripts/provision-e2e-permission-users.mjs</code> (credenziali solo in <code>.env</code> , non in codice app)

CI GitHub Actions

- `security.yml` : check RLS Supabase (set secrets `SUPABASE_URL` , `SUPABASE_ANON_KEY`)

Estendere pipeline con lint + test + build su PR (best practice repo rules).

17. Deploy produzione

Frontend

1. Imposta env build (`VESTIFLOW_API_BASE_URL` , Supabase anon, flags)
2. `npm run build`
3. Deploy `dist/vestiflow` su Firebase App Hosting
4. `CORS_ORIGINS` API deve includere dominio frontend

Backend

1. Railway (o altro): env da `api/.env.example`
2. `prisma migrate deploy` in release
3. Verifica `PLATFORM_ADMIN_EMAILS` con la tua email operatore
4. `SHOPIFY_APP_URL` = URL API pubblico HTTPS

Post-deploy smoke test

- ☐ Login tenant test
- ☐ Platform admin → Clienti → Nuovo cliente (staging)
- ☐ Platform admin → **Apri gestionale (assistenza)** su tenant test → banner + operazione magazzino → **Esci dall'assistenza**
- ☐ Profilo canale Shopify e TikTok su tenant test
- ☐ OAuth Shopify su tenant test
- ☐ Tenant Shopify: sync location → **Sedi attive in VestiFlow** → salva → verifica blocco UI
- ☐ Platform admin → Modifica cliente → **Concedi cambio sede** → cliente modifica e salva → riblocco
- ☐ Riduzione **Sedi incluse nel piano** su tenant con 2+ sedi attive → trim automatico
- ☐ OAuth TikTok Shop su tenant test (se abilitato)
- ☐ Upload foto profilo + avatar topbar
- ☐ Import catalogo + webhook
- ☐ Tenant **Solo gestionale**: POST retail-scans vendita + storno su variante test
- ☐ Tenant **Shopify**: Registra vendita + lista Vendite in sidebar; retail-scans + sync giacenze
- ☐ Tenant **TikTok** (se abilitato): retail-scans + push inventario
- ☐ Upload immagine prodotto
- ☐ Guida utente + guida tecnica admin

18. Sicurezza e checklist pre-release

- ☐ Nessun secret in git (`.env` gitignored)
- ☐ RLS attiva su tutte le tabelle (`npm run check:rls` se script root)
- ☐ `PLATFORM_ADMIN_EMAILS` limitato a operatori fidati

- ☐ CORS ristretto ai domini produzione
 - ☐ Token Shopify cifrati; revoca su disconnessione
 - ☐ MFA disponibile per admin negozio
 - ☐ Migration `0018_support_sessions`, `0021_tenant_location_licensing`, `0022_location_selection_lock` applicate in produzione
 - ☐ Sessioni assistenza tracciate in `support_sessions` (nessuna password condivisa per supporto)
 - ☐ Dipendenze: `npm audit` senza high/critical
 - ☐ Guide rigenerate se modificate
-

19. Troubleshooting tecnico

Problema	Azione
<code>isPlatformAdmin</code> false in UI	Verifica email in <code>PLATFORM_ADMIN_EMAILS</code> , ri-login
403 su <code>/admin/tenants</code>	Stesso controllo email lato API
CORS error	Aggiungi origin frontend a <code>CORS_ORIGINS</code>
JWT invalid	Allinea <code>SUPABASE_JWT_SECRET</code> con dashboard Supabase
Webhook non arrivano	URL tunnel/prod raggiungibile; HTTPS; webhook registrati
Import catalogo 429 / throttling Shopify	Attendi 1–2 min; non parallelizzare import; vedi \$9 limiti Shopify; controlla env <code>SHOPIFY_API_*</code>
API VestiFlow 429 (troppi click)	Limite 300 req/min/IP; chiedi al tenant di non ripetere azioni in loop
Immagini prodotto 404	Bucket <code>product-media</code> esiste ed è public
Avatar 404 / upload fallito	Bucket <code>user-avatars</code> esiste ed è public; env <code>SUPABASE_USER_AVATARS_BUCKET</code>
TikTok OAuth fallisce	Verifica <code>TIKTOK_*</code> env, callback URL pubblico HTTPS, app Partner Center attiva
Anon key legge dati	Critico — RLS mancante, fix migration immediato
500 su <code>POST .../support-session</code>	Migration <code>0018_support_sessions</code> non applicata — <code>npm run prisma:deploy</code> in <code>api/</code>
401 «Sessione assistenza non valida»	Sessione scaduta (>2 h) o chiusa; riavvia da Clienti. Verifica header inviato dall'interceptor
403 assistenza su tenant	Tenant contiene utente platform admin, oppure email operatore non in <code>PLATFORM_ADMIN_EMAILS</code>
403 «Selezione sedi bloccata» su PUT licensed	Concedi cambio sede da admin o verifica <code>locationSelectionLocked</code> / <code>locationSelectionChangeGranted</code>
Cliente vede tutte le location Shopify in magazzino	Non ha salvato Sedi attive o piano > sedi selezionate; verifica <code>licensedInVf</code>

20. Limitazioni note e roadmap

Area	Stato
Multi-store commerciali in un tenant	Non supportato — un shop = un tenant
Invito utenti / cambio ruolo self-service	Non in UI — solo provisioning iniziale + richiesta operatore
Sessione assistenza platform admin → tenant	Implementata — 2 h, read/write, audit <code>support_sessions</code> , banner UI
Sync vendite/clienti TikTok Shop	Non implementata — integrazione TikTok ancora parziale
Integrazione TikTok Shop (parità Shopify)	In sviluppo — oggi solo OAuth + push catalogo/giacenze
Bozze ordine Shopify (draft orders)	Non in scope — solo ordini confermati in Vendite
Rotazione sedi attive oltre limite piano	Bloccata lato API — solo Concedi cambio sede + salvataggio cliente entro <code>licensedLocationCount</code>
Location manuale senza Shopify	Parziale (location onboarding); sync Shopify consigliato; licensing via <code>licensedInVf</code>
Cassa / corrispettivi IT nativi	Non previsti — integrazione esterna; VF registra stock (retail-scans tutti i profili)
Vendita al banco	Implementata — <code>POST /inventory/retail-scans</code> , UI <code>/app/sales/register</code> , tutti i profili
Report server-side avanzati	In evoluzione
Coda bulk Shopify persistente (multi-tenant)	Non implementata — operazioni massicce sincrone HTTP
Notifiche email custom reset password	Config Supabase

Contatti

Manutenzione prodotto: **proprietario VestiFlow** (questo documento è il riferimento operativo interno).